

C# Project Report

Default Login Credentials

- Username: admin_0920
- Password: Admin@123

Key Features Implemented

- Time Table Management
 - Exam and Marks Management
 - Student Management
 - Staff Management
 - Teacher Management
 - Admin Management
 - Course Management
 - User ID and Password Management
-

Technologies Used

- **Database:** SQLite
- **Architecture:** MVC (Model-View-Controller)
- **Platform:** Windows Forms (C#)
- **Programming Paradigm:** Object-Oriented Programming (OOP)

Rules for using this application

There are some conditions for using this, namely that it is a company-related program and is designed to suit the Institute.

When you first log in, all the features for the admin will be displayed. You need to register an admin first, then you need to register the staff, then you need to register the coach and teachers. After registering the teachers, you need to fill the teachers in the courses.

After registering the information about the classrooms, you can start filling in the schedule. You can also fill in the schedules about the exams in the timetable.

The marks for the exams taken can be entered in the marks table.

After the course is completed, you only need to delete the course and it will automatically delete the students on the student table.

Soft delete method has been used without deleting stops and teachers and students

100049



Unicom TIC
Training & Innovation Centre

Time_table

exams

password_del

Student

marks

class and lab

Batch

Teacher detail

Staf

Admin

June 2025

Sun	Mon	Tue	Wed	Thu	Fri	Sat
25	26	27	28	29	30	31
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	1	2	3	4	5

Today: 6/23/2025

choos_date

corse_name

view

teacher

time

[9.00-10.30]

hall or lab

ADD_class UPDATE DILET

ADD_EXAM

Combo boxes have been used to minimize the use of text boxes as much as possible.

<<<This is a description for the timetable.



Unicom TIC
Training & Innovation Centre
Skills Today, Success Tomorrow

User_name -

password -

button2

The home page of the program



Unicom TIC
Training & Innovation Centre

Time_table

exams

password_del

Student

marks

class and lab

Batch

Teacher detail

Staf

Admin

user_name

admin_0920

User name already exists.

password

Admin@123

conform_password

Admin@123

Passwords match.

update

Unique travel and pick-up facilities included

```
-- Set AUTOINCREMENT starting values
INSERT OR REPLACE INTO sqlite_sequence (name, seq) VALUES ('Admin_table', 99999);
INSERT OR REPLACE INTO sqlite_sequence (name, seq) VALUES ('Course_table', 100050);
INSERT OR REPLACE INTO sqlite_sequence (name, seq) VALUES ('Teacher_table', 249999);
INSERT OR REPLACE INTO sqlite_sequence (name, seq) VALUES ('Staff_table', 104999);
INSERT OR REPLACE INTO sqlite_sequence (name, seq) VALUES ('Time_table', 10000000);
INSERT OR REPLACE INTO sqlite_sequence (name, seq) VALUES ('Student_table', 600001);
INSERT OR REPLACE INTO sqlite_sequence (name, seq) VALUES ('Exam_table', 1000);
```

Each user is given a unique ID number in the User table and the same ID numbers are found in the Personal table with them.

These IDs are passed as parameters to classes to perform operations.

```

1 reference
private void button13_Click(object sender, EventArgs e)
{
    teacher_detail_form teacher_Detail_Form = new teacher_detail_form(ROOT_ID);
    LoadForm_1(teacher_Detail_Form);
}

1 reference
private void button12_Click(object sender, EventArgs e)
{
    corse_form corse_Form = new corse_form(ROOT_ID);
    LoadForm_1(corse_Form);
}

1 reference
private void button11_Click(object sender, EventArgs e)
{
    class_and_labForm class_And_LabForm = new class_and_labForm(ROOT_ID);
    LoadForm_1(class_And_LabForm);
}

1 reference
private void button10_Click(object sender, EventArgs e)
{
    Time_table_form time_Table_Form = new Time_table_form(ROOT_ID);
    LoadForm_1(time_Table_Form);
}

1 reference
private void button9_Click(object sender, EventArgs e)
{
    students_Form students_Form=new students_Form(ROOT_ID);
    LoadForm_1(students_Form);
}

1 reference
private void button8_Click(object sender, EventArgs e)
{
    password_Form password_Form = new password_Form(ROOT_ID);
    LoadForm_1(password_Form);
}

1 reference
private void button7_Click(object sender, EventArgs e)
{

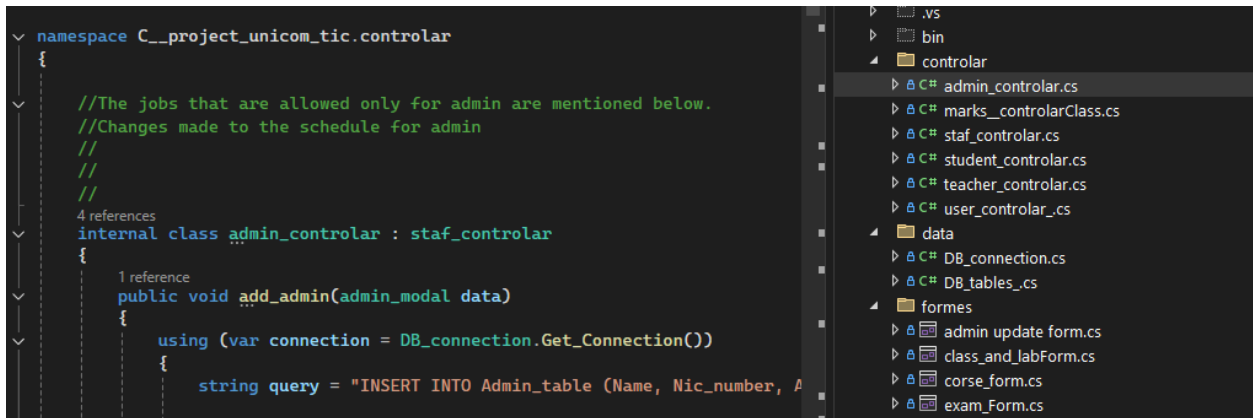
```

```

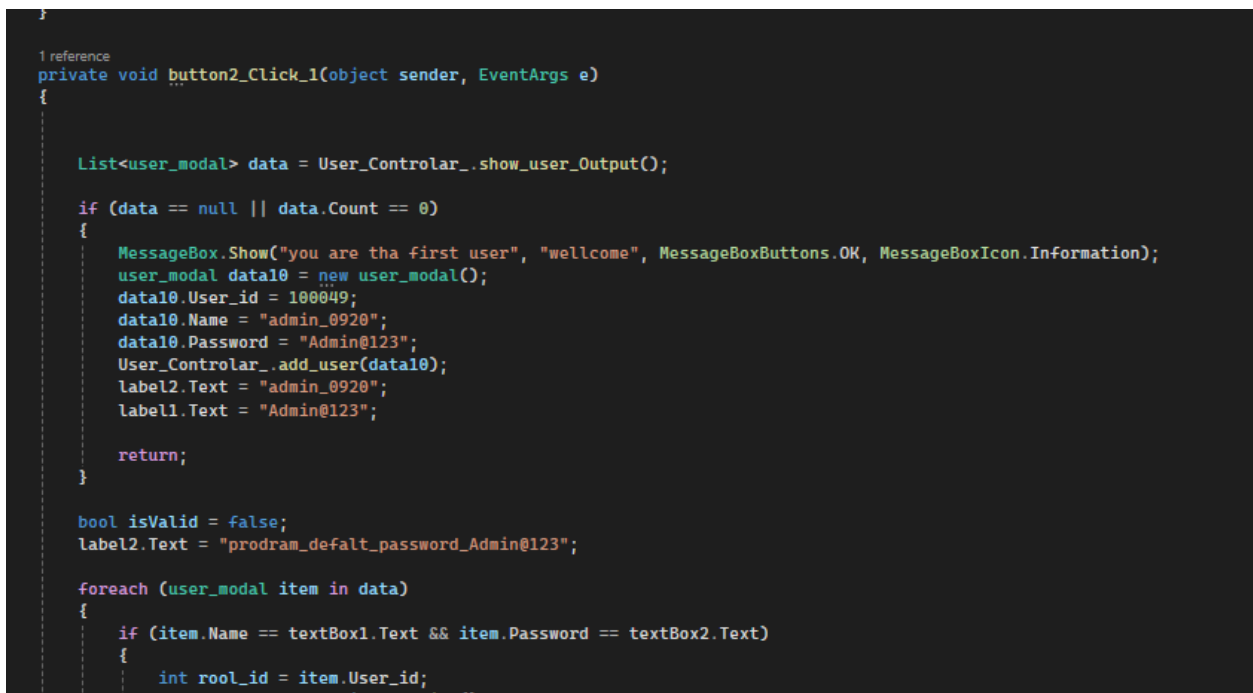
CREATE TABLE IF NOT EXISTS Exam_table (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL,
    Teacher_Id INTEGER NOT NULL,
    Course_Id INTEGER NOT NULL,
    Status TEXT NOT NULL,
    time_table_id INTEGER NOT NULL,
    Foreign Key (time_table_id) REFERENCES Time_table(Id),
    FOREIGN KEY (Course_Id) REFERENCES Course_table(Id),
    FOREIGN KEY (Teacher_Id) REFERENCES Teacher_table(Id),
    CHECK (Id > 1000 AND Id < 9999)
);

```

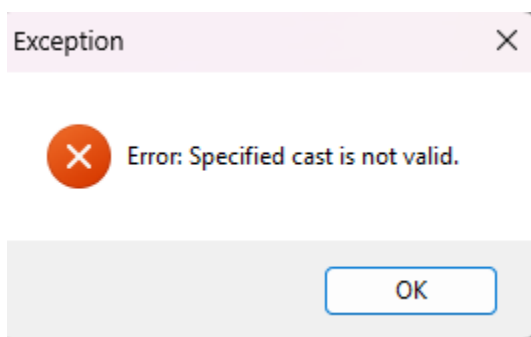
Teachers, students, staff, and admin will fall into different IT ranges.



All controllers are linked in a multi-level inheritance pattern, with a role-based relationship with them.



When a visitor logs into the program for the first time, the username and password are automatically configured and displayed in the labels.



Message box are now displayed for minor errors that occur during important operations and for some user errors.

Challenges Faced and Solutions

- The application was initially intended to be a large-scale system, but due to limited time and resources, the development had to be scaled down and completed quickly. This required prioritizing key features and managing the scope effectively to meet deadlines.
- Designing a flexible database structure using SQLite was a challenge, especially when handling relational data between students, teachers, courses, and exam records. Careful planning and normalization helped maintain consistency.
- Implementing user authentication and role-based access control was complex, especially in managing multiple user types (Admin, Teacher, Staff). To address this, a simplified login system with encrypted credentials and validation was used.
- One of the key challenges was managing form destruction and memory leaks in Windows Forms. Some forms were not properly disposed of, leading to performance issues. This was resolved by ensuring all opened forms were closed correctly and resources were released.
- Ensuring real-time data reflection in the UI after updates (e.g., adding a student or updating marks) required efficient use of data bindings and refresh methods, which took additional debugging and testing.
- UI responsiveness was another area of concern, especially with larger data sets. Loading times were optimized by filtering data and using background workers where necessary.
- Though many core features were implemented, some advanced functionalities like automated notifications, detailed analytics, and

responsive design were left out due to time constraints and can be included in future updates.